MCP 外部数据连接

让 AI 工具连接外部世界的标准协议——实战为主

第 2 周 · 第 6 节

讲师：黄佳

回顾上节：Hooks = 质量关卡

- **Hooks 解决了什么？** — 在关键节点自动执行检查（保存时 lint、提交前测试）
- **三种 Hook 类型：** pre-tool / post-tool / notification
- **核心价值：** 从“人记住规范”变成“系统执行规范”

- **但还有一个问题没解决：**
- 知识库需要外部数据 — GitHub 热门项目、RSS 订阅（HN/Lobsters/HF Papers）
- 这些数据源怎么接入？用什么方式最合理？

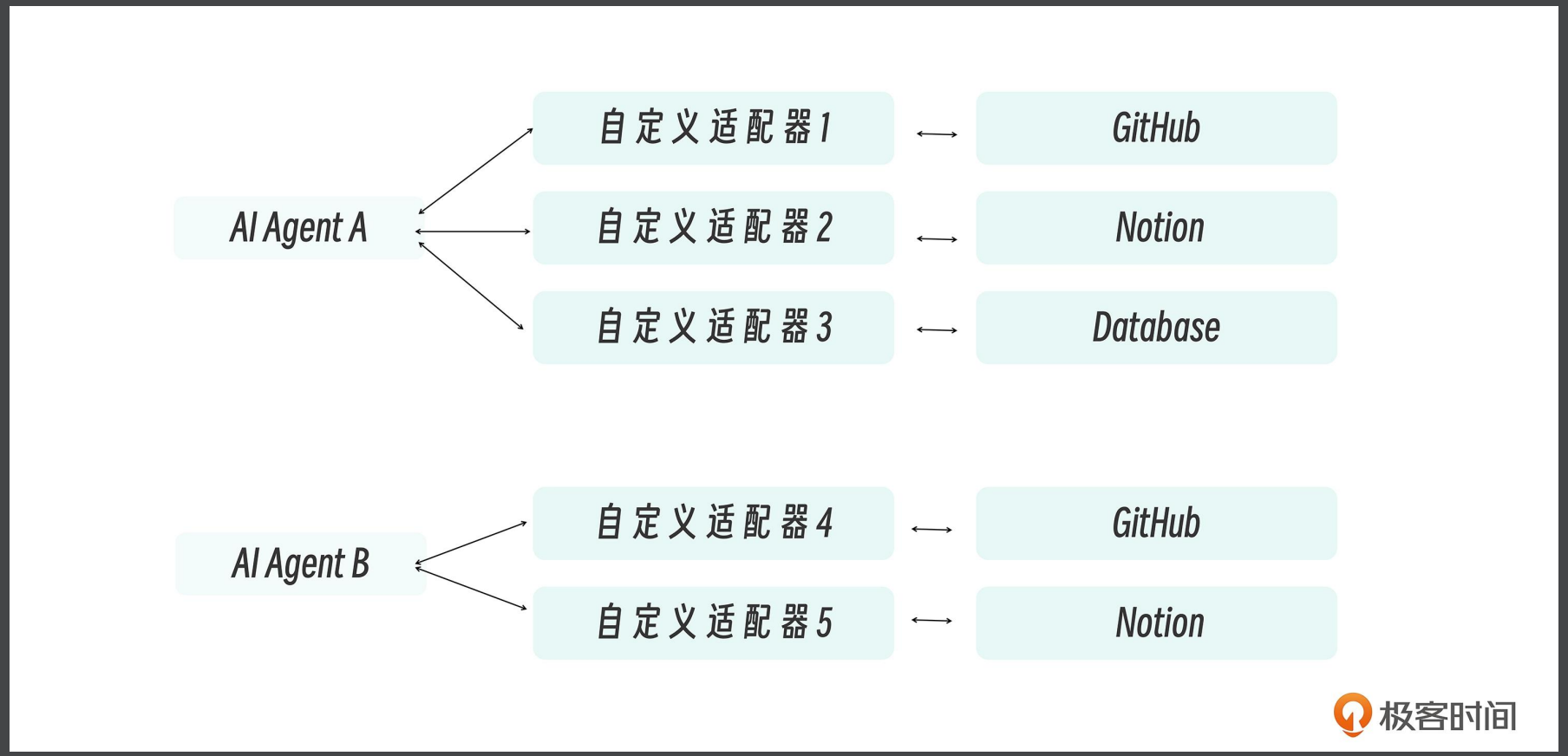
- **今天的目标：**
- 理解 MCP 协议的设计思想
- 掌握 Python pipeline 实战采集方案
- 学会免费采集 + 付费分析的成本优化策略

核心概念

MCP = AI 世界的 USB 接口

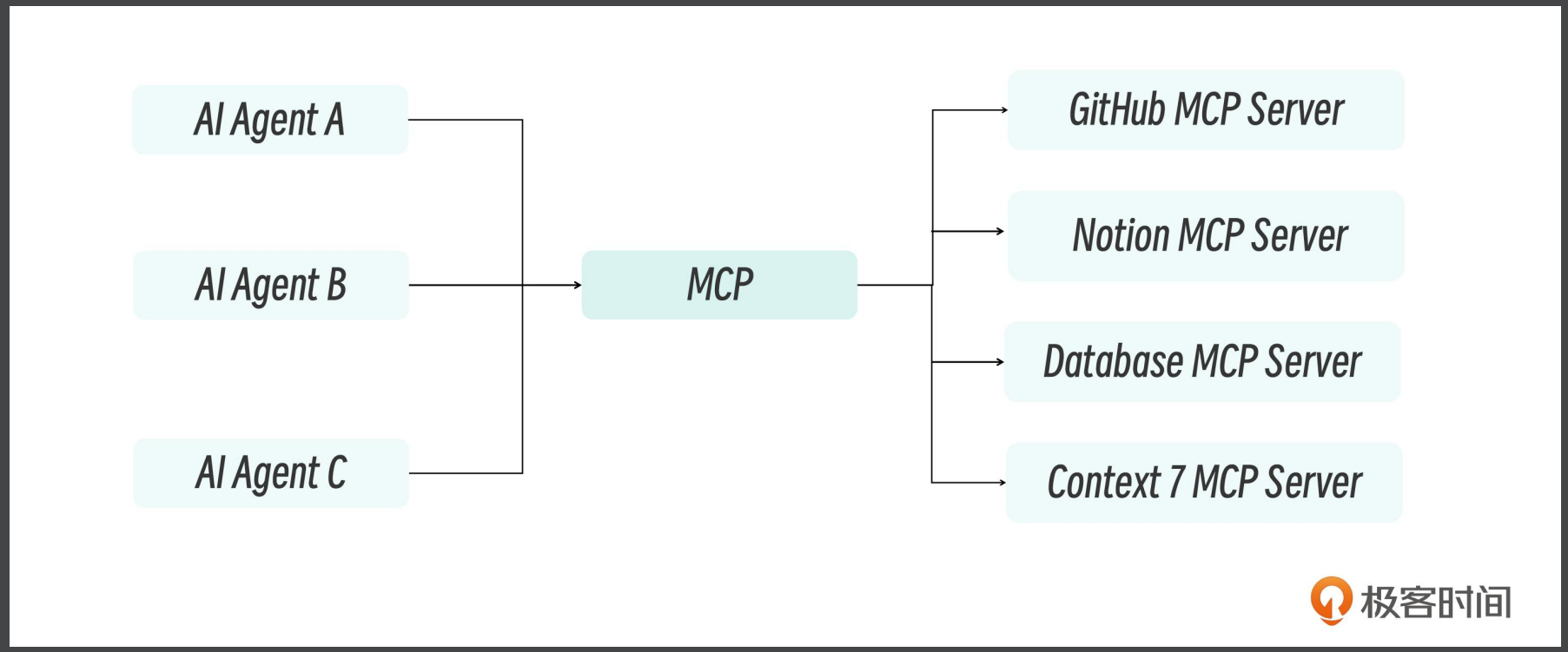
Model Context Protocol — 让 AI 工具连接外部世界的标准协议

没有 MCP 之前：M x N（每个都要自己适配）



2 个 Agent x 3 个服务 = 5 个自定义适配器

有了 MCP：M + N（标准协议统一）



M 个 Agent + N 个 Server = M + N 个对接

MCP 架构：Client ↔ Server

OpenCode (MCP Client)

用户对话 → AI 决策 → 选择工具 → 调用 MCP Server

MCP Server
GitHub API

MCP Server
Databas（SQLite）

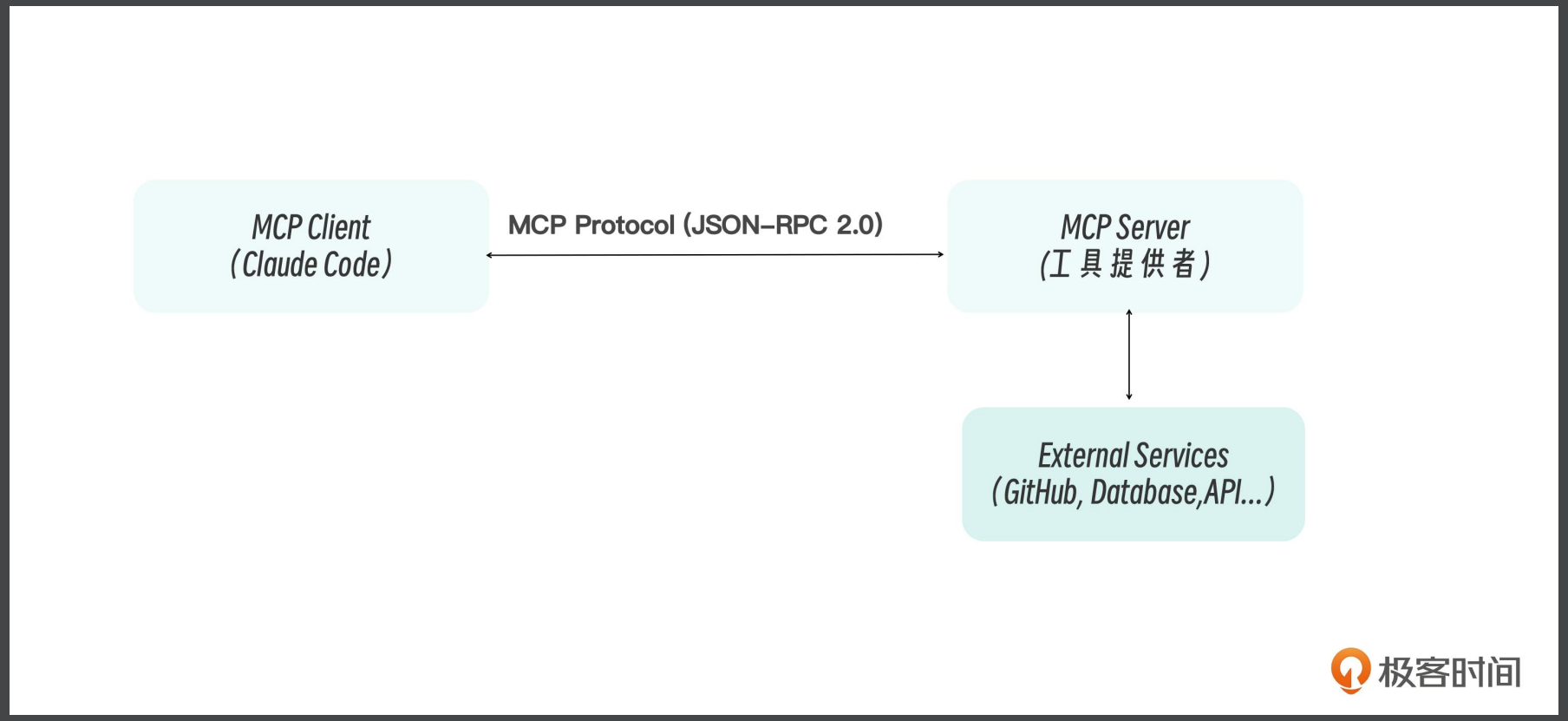
MCP Server
Search
Engine

- 协议标准：JSON-RPC 2.0
- 核心能力：Tools（工具调用）/ Resources（数据读取）/ Prompts（提示模板）

MCP 三种传输方式

传输方式	协议	适用场景	示例
stdio	标准输入输出	本地进程通信 (最常用)	本地 SQLite Server 本地文件系统 Server
SSE	Server-Sent Events (HTTP 流式)	远程服务 需要实时推送	远程搜索引擎 实时数据监控
HTTP	标准 HTTP 请求	简单的远程调用 无状态请求	REST API 封装 云服务集成

MCP 架构：Client – Server – External



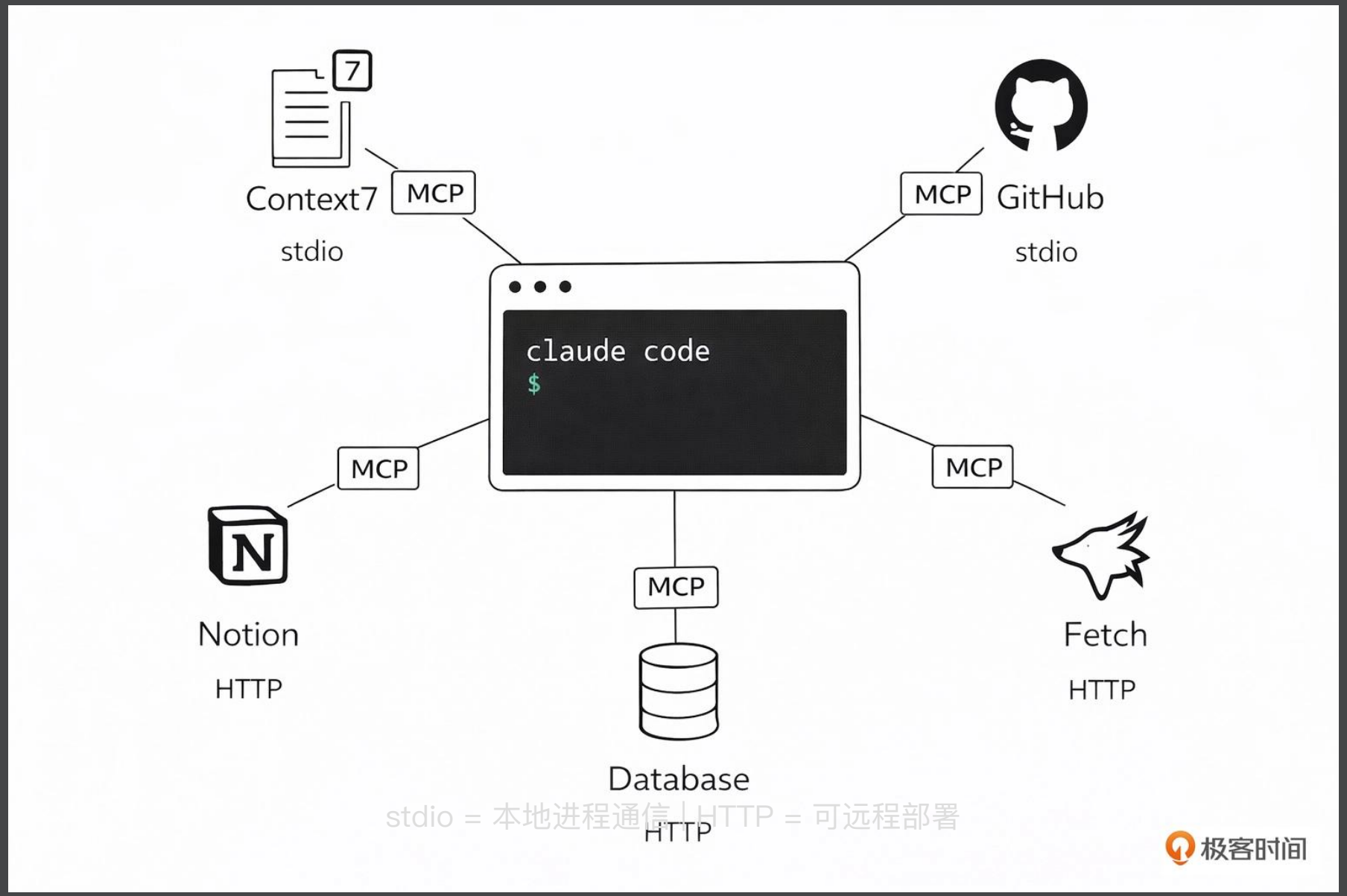
MCP Server = 本地翻译官，不是云端服务器

在 opencode.json 中配置 MCP Server

opencode.json | json

```
// opencode.json — 项目根目录
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-github"
      ],
      "env": {
        "GITHUB_TOKEN": "ghp_xxxxxxxxxxxx"
      }
    },
    "sqlite": {
      "command": "uvx",
      "args": [
        "mcp-server-sqlite",
        "--db-path", "./data/knowledge.db"
      ]
    },
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    }
  }
}
```

Claude Code 的 MCP 全景



实战选择：MCP vs Python 直接调 API

MCP 方案

AI 动态决策用哪个工具
适合交互式探索场景
你说"搜一下"，AI 自己选 Server

优势：灵活、标准化、可扩展
劣势：每次调用都消耗 Token
采集 30 条 = 30 次 AI 决策
成本高、速度慢、不可控

适合：需要 AI 判断的场景

Python Pipeline 方案

代码直接调 API，不需要 AI
适合确定性流水线场景
urllib/requests 调 GitHub API

优势：零 Token 成本、快速、可控
劣势：需要写代码、不够灵活
采集 30 条 = 1 次 API 调用
纯 Python，零 AI 成本

适合：知识库采集这种固定流程

Python 自动化架构：pipeline.py 编排全流程

pipeline.py（流水线编排器）

step 1	step 2	step 3	step 4
GitHub API 采集 纯 API 零成本	RSS 执行操作 纯 API 零成本	AI 深度分析 调用 LLM 有成本	整理入库 去重+格式 零成本
raw/github.json	raw/rss.json	analyzed/*.json	articles/*.md
免费步骤	免费步骤	付费步骤	免费步骤

讨论和思考：Skill + CLI 脚本 = MCP 的 80% 价值?

Skill + Bash 方案（20 分钟）

SKILL.md + scripts/check_deploy.sh
  10 行 curl + jq 搞定

Skill 提供：知识（什么时候用、怎么分析）
Bash 提供：执行（curl 调 API、jq 解析）

组合 = 知识 + 执行 + 自动发现
= MCP 大部分价值

MCP 方案（半天到一天）

写 Server（100+ 行 Node.js/Python）
  实现 tools/list, tools/call 协议
  注册、调试、超时、重启

MCP 不可替代的场景：
  * 有状态长连接（连接池、事务）
  * 复杂认证（OAuth2、mTLS）
  * 同一服务 10+ 个工具
  * 跨客户端复用（Cursor/Copilot）

LLM 的理解能力正在消解结构化协议的必要性
Claude 能读 stderr、能解析非结构化输出 = JSON Schema 约束的边际价值下降

model_client.py — 统一模型客户端

scripts/model_client.py | python

```python
import urllib.request, urllib.parse, json, os

PROVIDERS = {
    "deepseek": {
        "base_url": "https://api.deepseek.com/v1",
        "model": "deepseek-chat",
        "env_key": "DEEPSEEK_API_KEY",
    },
    "qwen": {
        "base_url":
"https://dashscope.aliyuncs.com/compatible-mode/v1",
        "model": "qwen-plus",
        "env_key": "QWEN_API_KEY",
    },
    "glm": {
        "base_url": "https://open.bigmodel.cn/api/paas/v4",
        "model": "glm-4-flash",
        "env_key": "GLM_API_KEY",
    },
}
```

scripts/model_client.py | python

```python
def quick_chat(prompt: str, provider: str = "deepseek") ->
str:
    """一套代码调所有国产模型（都兼容 OpenAI API）"""
    cfg = PROVIDERS[provider]
    api_key = os.environ[cfg["env_key"]]
    url = f'{cfg["base_url"]}/chat/completions'
    body = json.dumps({"model": cfg["model"],
        "messages":
[{"role":"user","content":prompt}]}).encode()
    req = urllib.request.Request(url, data=body, headers={
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"})
    resp = json.load(urllib.request.urlopen(req))
    return resp["choices"][0]["message"]["content"]
```

pipeline.py Step 1 — GitHub API 采集

scripts/model_client.py | python

```python
import urllib.request, json, datetime

GITHUB_API = "https://api.github.com/search/repositories"

def collect_github(limit: int = 30, min_stars: int = 100):
    """Step 1: 采集 GitHub 热门项目（纯 API，零 LLM 成本）"""
    since = (datetime.date.today()
             - datetime.timedelta(days=days)).isoformat()
    queries = [
        f"created:>{since} stars:>{min_stars} topic:ai",
        f"created:>{since} stars:>{min_stars} topic:llm",
        f"created:>{since} stars:>{min_stars} topic:agent",
    ]
    items = []
```

scripts/model_client.py | python

```python
for q in queries:
    url = f"{GITHUB_API}?q={urllib.parse.quote(q)}"\
          f"&sort=stars&order=desc&per_page=30"
    req = urllib.request.Request(url, headers={
        "Accept": "application/vnd.github.v3+json"})
    data = json.load(urllib.request.urlopen(req))
    items.extend(data.get("items", []))
# 去重 + 提取关键字段
seen, result = set(), []
for r in items:
    if r["id"] not in seen:
        seen.add(r["id"])
        result.append({"name": r["full_name"],
            "stars": r["stargazers_count"],
            "url": r["html_url"],
            "desc": r["description"] or ""})
return result[:30]  # 最多 30 条
```

pipeline.py Step 2 — RSS 订阅采集

scripts/pipeline.py | python

```python
import xml.etree.ElementTree as ET

RSS_FEEDS = [
    "https://hnrss.org/best?count=20",
    "https://lobste.rs/rss",
    "https://huggingface.co/papers/rss",
]

def collect_rss(limit: int = 30):
    """Step 2: 采集 RSS 订阅（纯 API，零 LLM 成本）"""
    items = []
    for feed_url in RSS_FEEDS:
        xml_data = urllib.request.urlopen(feed_url).read()
        root = ET.fromstring(xml_data)
        for entry in root.iter("item"):
            items.append({
                "title": (entry.findtext("title") or "").strip(),
                "url":   (entry.findtext("link") or "").strip(),
                "desc":  (entry.findtext("description") or "")[:500],
                "date":  (entry.findtext("pubDate") or "").strip(),
                "source": feed_url.split("/")[2],
            })
    return items[:limit]
```

pipeline.py Step 3 — AI 深度分析（调用 LLM）

scripts/model_client.py | python

```python
from model_client import quick_chat

ANALYZE_PROMPT = """你是技术分析师。对以下技术内容做深度分析：

## 内容
{content}

## 要求
1. 一句话中文摘要（≤50 字）
2. 技术亮点（2-3 个，用事实说话）
3. 相关性评分（1-10，9-10 改变格局，7-8 直接有帮助）
4. 标签（从 [AI, LLM, Agent, DevTool, Framework, ...] 选 2-5 个）

返回 JSON 格式。"""
```

scripts/model_client.py | python

```python
def step_analyze(items: list, provider: str = "deepseek"):
    """Step 3: AI 深度分析（调用 LLM，有成本）"""
    results = []
    for item in items:
        content = json.dumps(item, ensure_ascii=False)
        prompt = ANALYZE_PROMPT.format(content=content)
        analysis = quick_chat(prompt, provider=provider)
        try:
            results.append(json.loads(analysis))
        except json.JSONDecodeError:
            results.append({"raw": analysis})
    return results
```

pipeline.py Step 4 — 整理入库（去重 + 格式化 + 写文件））

scripts/model_client.py | python

```python
import os, hashlib

def step_organize(analyzed: list, output_dir: str = "articles"):
    """Step 4: 整理入库（纯 Python，零 LLM 成本）"""
    os.makedirs(output_dir, exist_ok=True)

    # 读取已有文件做去重
    existing = set()
    for f in os.listdir(output_dir):
        existing.add(f.split("_")[0])  # hash prefix

    written = 0
    for item in analyzed:
        # 用 URL 生成唯一 ID（去重键）
        uid = hashlib.md5(
            item.get("url", "").encode()
        ).hexdigest()[:8]
```

scripts/model_client.py | python

```python
        if uid in existing:
            continue  # 跳过已存在的条目

        filename = f"{uid}_{item.get('title', 'untitled')}.json"
        filepath = os.path.join(output_dir, filename)
        with open(filepath, "w", encoding="utf-8") as f:
            json.dump(item, f, ensure_ascii=False, indent=2)
        written += 1

    print(f"Written {written} new articles, "
          f"skipped {len(analyzed) - written} duplicates")
    return written
```

rss_reader.py — 支持任意 RSS 数据源

scripts/rss_reader.py | python

```python
import xml.etree.ElementTree as ET
import urllib.request

RSS_FEEDS = {
    "hn_best":   "https://hnrss.org/best?count=20",
    "lobsters":  "https://lobste.rs/rss",
    "hf_papers": "https://huggingface.co/papers/rss",
    # 自定义 RSS 源可以随时添加
}

def fetch_rss(feed_name: str) -> list[dict]:
    """通用 RSS 读取器（纯 Python，零 LLM 成本）"""
    url = RSS_FEEDS[feed_name]
    xml_data = urllib.request.urlopen(url).read()
    root = ET.fromstring(xml_data)

    items = []
    for item in root.iter("item"):
        items.append({
            "title": (item.findtext("title") or "").strip(),
            "url":   (item.findtext("link") or "").strip(),
            "desc":  (item.findtext("description") or "")[:500],
            "date":  (item.findtext("pubDate") or "").strip(),
            "source": feed_name,
        })
    return items
```

多源采集策略：GitHub + RSS = 双路采集

- **三路数据源，全部免费：**
- GitHub Search API — AI/LLM/Agent 热门项目（认证后 30 次/分钟）
- RSS Feeds — 任意订阅源（HN Best、Lobsters、HF Papers...）
-
- **免费 vs 付费分离（核心设计原则）：**
- Step 1–2：纯 API 采集 → 零成本（Python urllib）
- Step 3：AI 深度分析 → 有成本（DeepSeek ~0.02 元/30 条）
- Step 4：整理入库 → 零成本（Python 字符串处理）
-
- **每日运行成本估算：**
- 采集 60 条（GitHub 30 + RSS 30） = 免费
- AI 分析 60 条 ≈ 0.04 元（DeepSeek） / 0.2 元（Qwen）
- 一个月 ≈ 1.2 元 ~ 6 元，比一杯咖啡便宜

Pipeline CLI — 灵活控制流水线

scripts/pipeline.py --help | bash

```bash
# 只采集 GitHub（免费）
python pipeline.py --sources github --limit 20

# 采集 GitHub + RSS（免费）
python pipeline.py --sources github,rss --limit 20

# 完整流水线（采集 + 分析 + 入库）
python pipeline.py --sources github,rss --limit 20 --analyze

# 切换模型（默认 DeepSeek，可选 qwen/glm/kimi）
python pipeline.py --sources github,rss --limit 20 --analyze --provider qwen

# 干跑模式（只打印计划，不实际执行）
python pipeline.py --sources github,rss --limit 20 --dry-run

# 查看所有可用数据源
python pipeline.py --list
  Source: github （免费）GitHub API 采集
  Source: rss    （免费）RSS 订阅采集
  Step:   analyze（付费）AI 深度分析
  Step:   organize（免费）整理入库
```

本节作业

任务 1：编写 model_client.py 统一模型客户端

参考课上代码，实现 PROVIDERS 字典和 quick_chat() 函数。至少配置 2 个国产模型（DeepSeek + Qwen），用真实 API Key 测试。验证方式：调用 quick_chat('你好', provider='deepseek') 和 quick_chat('你好', provider='qwen') 都能正常返回。

任务 2：编写 pipeline.py 流水线

实现 collect_github(limit) 和 collect_rss(limit) 两个免费采集函数，以及 step_analyze() 和 step_organize() 两个处理函数。先跑 --sources github,rss --limit 20 确认采集正常，再加 --analyze 跑完整流水线。检查 articles/ 目录是否生成了去重后的 JSON 文件。

任务 3：添加 RSS 数据源

实现 rss_reader.py，至少配置 2 个 RSS 源。推荐：hn_best（HN 精选）+ hf_papers（HuggingFace 论文）。将 RSS 采集的数据也接入 pipeline 的 Step 3 分析流程。（可选挑战）添加一个自己常读的技术博客 RSS。

实操 4：MCP 知识库 Server 实战

给知识库写一个 MCP Server，让 OpenCode 通过自然语言搜索你的文章

mcp_knowledge_server.py

search_articles(keyword)
  按关键词搜索标题、摘要、标签

get_article(article_id)
  按 ID 获取文章完整内容

knowledge_stats()
  文章总数、来源分布、热门标签

协议：JSON-RPC 2.0 over stdio
依赖：零（纯 Python 标准库）

opencode.json 配置

```
{ "$schema": "...opencode.ai...",
  "mcp": {
    "knowledge": {
      "type": "local",
      "command": ["python3",
        "mcp_knowledge_server.py"],
      "enabled": true
    } } }
```

验证 & 体验

```
$ opencode mcp list

> 搜索关于 RAG 的文章
> 知识库里哪些标签最热门?
> 给我看 github-20260326-002 的详情
```

MCP 的核心价值：把你的数据变成 AI 可调用的工具，用对话代替命令行

工程思想总结卡

- **MCP = 标准化连接协议**
- Client ↔ Server 架构，OpenCode 内置支持
- 三种传输：stdio（本地） / SSE（远程流式） / HTTP（标准请求）
- 适合需要 AI 动态决策的交互式场景
- **Python Pipeline = 实战利器**
- 确定性流程用代码，不需要 AI 决策
- model_client.py 统一模型客户端，一键切换国产模型
- pipeline.py 四步流水线：采集 → 采集 → 分析 → 入库
- **免费采集 + 付费分析 = 成本最优**
- Step 1–2 纯 API 零成本，Step 3 调 LLM 有成本
- 每日 60 条分析 ≈ 0.04 元，每月不到一杯咖啡
- 核心原则：能用代码解决的就不用 AI

本节小结

MCP = 标准化外部工具接入协议

MCP Server 提供能力，MCP Client 消费能力

一次接入，所有支持 MCP 的工具都能用

GitHub / 数据库 / API → 统一通过 MCP 连接

下一节：第 7 节 · CI/CD 持续集成 — 让流水线全自动运行